

Basic Maths

La mayoría de algoritmos de divisibilidad requieren revisar todos los números, lo cual implica una complejidad de $O(n)$. Para evitar esta complejidad, podemos tomar en cuenta lo siguiente:

Supongamos que n tiene un divisor d y un k tal que $n = d \cdot k$. Si intentamos que tanto d como k sean mayores que $\sqrt{n}$ ($d > \sqrt{n}$ y $k > \sqrt{n}$), entonces al calcular su producto veríamos que $d \cdot k > \sqrt{n} \cdot \sqrt{n}$, lo cual implica $d \cdot k > n$. Esto es una contradicción, por lo cual se puede afirmar que en cada pareja de divisores, al menos uno debe ser $\leq \sqrt{n}$. Esto permite reducir la complejidad de $O(n)$ a $O(\sqrt{n})$.

Divisores

Dado un número n , para hallar todos sus divisores con complejidad $O(\sqrt{n})$, podemos iterar hasta la raíz y notar que si i es un divisor, entonces n/i también lo es.

```
1 vector<int>div(int n){
2     vector<int>d;
3     for(int i=1;(long long)i*i<=n;i++){
4         if(n%i==0){
5             d.push_back(i);
6             if(i!=n/i)d.push_back(n/i);
7         }
8     }
9     sort(d.begin(),d.end());
10    return d;
11 }
```

Número de Divisores

Usando la factorización en números primos de n , donde p_i es un número primo y e_i su exponente, expresamos n como:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdot p_3^{e_3} \cdots p_k^{e_k} \quad (1)$$

El número de divisores $d(n)$ se obtiene mediante todas las combinaciones posibles de los exponentes de sus factores primos (desde 0 hasta e_i):

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdot (e_3 + 1) \cdots (e_k + 1) \quad (2)$$

Si tomamos un número con dos factores primos distintos, $n = p_1^{e_1} \cdot p_2^{e_2}$, podemos organizar todos sus divisores en una tabla de doble entrada. Cada celda representa un divisor único:

$\times$	1	p_2	p_2^2	$\dots$	$p_2^{e_2}$
1	1	p_2	p_2^2	$\dots$	$p_2^{e_2}$
p_1	p_1	$p_1 p_2$	$p_1 p_2^2$	$\dots$	$p_1 p_2^{e_2}$
p_1^2	p_1^2	$p_1^2 p_2$	$p_1^2 p_2^2$	$\dots$	$p_1^2 p_2^{e_2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$p_1^{e_1}$	$p_1^{e_1}$	$p_1^{e_1} p_2$	$p_1^{e_1} p_2^2$	$\dots$	$p_1^{e_1} p_2^{e_2}$

```

1 #define ll long long
2 ll cntd(ll n){
3     ll cnt=1;
4     for(ll i=2;i*i<=n;i++){
5         if(n%i==0){
6             ll exp=0;
7             while(n%i==0){
8                 n/=i;
9                 exp++;
10            }
11            cnt*=(exp+1);
12        }
13    }
14    if(n>1) cnt*=2;
15    return cnt;
16 }

```

Para múltiples consultas en un rango acotado (10^6), podemos aplicar una técnica similar a la Criba de Eratóstenes para precalcular el número de divisores de todos los números en $O(M \log M)$:

```

1 const int M=1e6;
2 vector<int> p(M+1,0);
3 void pre(){
4     for(int i=1;i<=M;i++){
5         for(int j=i;j<=M;j+=i){
6             p[j]++;
7         }
8     }
9 }

```

Suma de Divisores

Para calcular la suma de todos los divisores de n , denotada como $\sigma(n)$, utilizamos de nuevo su factorización prima:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k} \quad (3)$$

Considerando la tabla de divisores anterior, la suma de todos los elementos de la matriz se puede obtener multiplicando la suma de las opciones de cada factor

primo. La suma de las potencias de un primo p desde 0 hasta e es una progresión geométrica:

$$\sigma(p^e) = \sum_{k=0}^e p^k = 1 + p + p^2 + \cdots + p^e = \frac{p^{e+1} - 1}{p - 1} \quad (4)$$

Dado que $\sigma(n)$ es una función multiplicativa, la suma total de los divisores es el producto de las sumas individuales de cada factor primo:

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{e_i+1} - 1}{p_i - 1} \quad (5)$$

```

1  #define ll long long
2  ll sumd(ll n) {
3      ll tot=1;
4      for(ll i=2; i*i<=n; i++){
5          if(n%i==0){
6              int e=0;
7              while(n%i==0){
8                  e++;
9                  n/=i;
10             }
11             ll cs=0;
12             ll p=i;
13             for(int j=0; j<=e; j++){
14                 cs+=p;
15                 p*=i;
16             }
17             tot*=cs;
18         }
19     }
20     if(n>1) tot*=(1+n);
21     return tot;
22 }

```

GCD (Greatest Common Divisor)

Sean dos números enteros a y b , podemos hallar el valor de $\gcd(a, b)$ utilizando el Algoritmo de Euclides. Este algoritmo reduce el valor de a (el máximo) usando módulos hasta que b sea 0. En cada iteración, los valores cambian: a pasa a ser b (divisor) y b pasa a ser $a \bmod b$ (residuo). El algoritmo termina y halla el valor cuando $b = 0$.

```

1  // Enfoque Recursivo
2  int gcd(int a, int b){
3      if(b==0) return a;
4      return gcd(b, a%b);
5  }
6  // Enfoque Iterativo
7  int gcd(int a, int b){
8      while(b){

```

```

9      a%=b;
10     swap(a,b);
11 }
12 return a;
13 }

```

Algunas propiedades fundamentales de gcd a considerar son:

1. **Conmutatividad:** $\gcd(a, b) = \gcd(b, a)$.
2. **Asociatividad:** $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$. (Útil para hallar el GCD de un arreglo de números).
3. **Distributividad:** $\gcd(m \cdot a, m \cdot b) = m \cdot \gcd(a, b)$ o $\gcd(a, \text{lcm}(b, c)) = \text{lcm}(\gcd(a, b), \gcd(a, c))$.
4. Sea $g = \gcd(a, b)$, todos los divisores de g también son divisores comunes de a y b .
5. Casos con cero: $\gcd(a, 0) = a$ (asumiendo $a > 0$). Si ambos son cero, $\gcd(0, 0)$ es indefinido matemáticamente (aunque en programación suele devolver 0).
6. Sea $g = \gcd(a, b)$. Si a divide al producto $b \cdot c$, entonces se cumple que $\frac{a}{g}$ divide a c .
7. Sea $g = \gcd(a, b)$, entonces se cumple que $\gcd\left(\frac{a}{g}, \frac{b}{g}\right) = 1$. Es decir, al dividir ambos números por su máximo común divisor, los resultados siempre son coprimos.
8. Para cualquier entero m , se cumple la propiedad fundamental del Algoritmo de Euclides: $\gcd(a, b) = \gcd(a + m \cdot b, b)$.
9. Si m es un divisor común de a y b , se cumple que: $\gcd\left(\frac{a}{m}, \frac{b}{m}\right) = \frac{\gcd(a, b)}{m}$.
10. Sean a_1 y a_2 números coprimos ($\gcd(a_1, a_2) = 1$), se cumple que: $\gcd(a_1 \cdot a_2, b) = \gcd(a_1, b) \cdot \gcd(a_2, b)$.
11. Desde la factorización prima, si $a = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$ y $b = p_1^{f_1} \cdot p_2^{f_2} \cdots p_k^{f_k}$, entonces: $\gcd(a, b) = p_1^{\min(e_1, f_1)} \cdot p_2^{\min(e_2, f_2)} \cdots p_k^{\min(e_k, f_k)}$.
12. Para cualesquiera enteros a y b , y una base n , se cumple que: $\gcd(n^a - 1, n^b - 1) = n^{\gcd(a, b)} - 1$.
13. Sean $P_0 = (0, 0)$ y $P_1 = (x, y)$. El número de segmentos en los que se divide la línea que une ambos puntos por coordenadas enteras es $\gcd(|x|, |y|)$. El número total de puntos con coordenadas enteras (lattice points) en ese segmento, incluyendo los extremos, es $\gcd(|x|, |y|) + 1$.

LCM (Least Common Multiple)

El Mínimo Común Múltiplo de dos enteros a y b , denotado como $\text{lcm}(a, b)$, es el entero positivo más pequeño que es divisible simultáneamente por a y por b .

La forma más eficiente de calcular el LCM es utilizando su estrecha relación matemática con el Máximo Común Divisor (GCD). La fórmula fundamental que los conecta es:

$$\text{lcm}(a, b) = \frac{a \cdot b}{\text{gcd}(a, b)} \quad (6)$$

```
1 typedef long long ll;
2 ll lcm(ll a, ll b){
3     if(a==0 || b==0) return 0;
4     return (a/gcd(a, b))*b;
5 }
```

Algunas propiedades fundamentales de lcm a considerar son:

1. **Conmutatividad:** $\text{lcm}(a, b) = \text{lcm}(b, a)$.
2. **Asociatividad:** $\text{lcm}(a, \text{lcm}(b, c)) = \text{lcm}(\text{lcm}(a, b), c)$. (Se usa para calcular el LCM de un arreglo completo iterando y acumulando el resultado).
3. **Distributividad:** $\text{lcm}(m \cdot a, m \cdot b) = m \cdot \text{lcm}(a, b)$ o $\text{lcm}(a, \text{gcd}(b, c)) = \text{gcd}(\text{lcm}(a, b), \text{lcm}(a, c))$.
4. Para cualesquiera dos enteros positivos, $\text{gcd}(a, b) \cdot \text{lcm}(a, b) = a \cdot b$.
5. Desde la factorización prima, si $a = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$ y $b = p_1^{f_1} \cdot p_2^{f_2} \cdots p_k^{f_k}$, entonces a diferencia del GCD (que toma el mínimo), el LCM toma el máximo exponente para cada primo:
$$\text{lcm}(a, b) = p_1^{\max(e_1, f_1)} \cdot p_2^{\max(e_2, f_2)} \cdots p_k^{\max(e_k, f_k)}.$$
6. Si a divide a b , entonces $\text{lcm}(a, b) = b$.
7. Cualquier múltiplo común de a y b es garantizadamente un múltiplo de $\text{lcm}(a, b)$.

Pequeño Teorema de Fermat

Sean p (numero primo) y a coprimos ($\text{gcd}(a, p) = 1$) se cumple:

$$a^{p-1} \equiv 1 \pmod{p} \quad (7)$$